Grading and Feedback

Course: REI603M The AI Lifecycle
Assignment: Assignment 6 – Progress, Monitoring & Next Steps
Students: Jóhannes Reykdal Einarsson, Sævar Breki Snorrason, Sölvi Santos
Project: Ratatoskur – iOS Math Tutor with Handwritten Input

—

Grade: 10/10

All requirements met. Three progress menu items with meaningful work, live demo, 11-slide presentation covering all prescribed content, 5 planned features with rationale, and Git repository links provided. Good team coordination with clear ownership per member.

—

Checklist

Progress Menu (at least 3 items):

- [x] Extend Observability & Monitoring (Sævar) – Improved Langfuse dashboard beyond A4, added token efficiency analysis, latency/usage trend tracking, thumbs up/down feedback with optional comments.
- [x] UI/UX & User Experience (Sölvi) – Added streaming responses, improved loading states, enhanced interface and user flow.
- [x] Data Collection & Analytics (Jóhannes) – Dashboard for user behaviour and failures, feedback summarization, weekly reporting concept feeding into GitHub issues.

Monitoring & Observability:

- [x] Monitoring beyond A4 – Moved from single-call logging to structured Langfuse tracing with token efficiency and usage patterns.
- [x] Dashboard with metrics – Langfuse dashboard with system-level and user-level observability.
- [x] Data insights discussed – Acknowledged the challenge of no real users. Showed early trends: mode latency ordering (hint < check_solution < reveal), latency growing linearly with total tokens, thought tokens dominating usage.

Feature Plan:

- [x] At least 5 features – User-based homepage statistics, error bank, latency reduction, error-based problem generation, anonymous data collection.
- [x] Each feature has rationale.

Presentation:

- [x] Slides cover prescribed structure (11 slides, all content present).
- [x] Live demo shown.
- [x] Slides submitted by deadline.
- [x] Git repository links provided (backend + frontend).

Deductions: None

Final grade: 10 – 0 = 10/10

—

Feedback on Progress Items

Observability & Monitoring – Moving from logging single calls to structured tracing with token efficiency analysis is a meaningful step. The observation that thought tokens dominate usage is useful – it suggests exploring models without thought tokens (like GPT-5.3 instant) could cut costs significantly without degrading quality for simpler operations like hint generation.

UI/UX – Adding streaming and improved loading states directly improves perceived performance, which matters when LLM calls take seconds. Good design sense to work on this.

Data Collection & Analytics – The weekly reporting concept feeding into GitHub issues is a practical approach to closing the feedback loop. The "What the Data Tells You" slide was thin, which is understandable without real users. For the final submission, generate synthetic data from team testing sessions (each member does 20-30 practice sessions at different skill levels) to demonstrate the analytics pipeline working end-to-end.

—

Feedback on Planned Features

1. **User-based statistics on homepage** – Good for personalization. Show success rate by topic, recent activity, and suggested next problem type based on weak areas.
2. **Error bank** – Cataloguing errors so users can review their mistakes is a strong learning feature. Classify errors by type (sign error, wrong rule, computational mistake, conceptual) to make the bank useful for targeted practice.
3. **Lower latency** – Check whether the model's thought tokens are needed for all operations. Hint generation likely needs less reasoning than solution checking. Using a cheaper/faster model for hints could cut latency for the most common operation.
4. **Error-based problem generation** – The key challenge is classifying error types consistently. Have the LLM return structured JSON (`{error_type, error_step, correct_approach}`) when checking solutions, then aggregate error types per user to find patterns.
5. **Anonymous data collection** – Handwritten math images are biometrically identifiable (handwriting is a personal characteristic). For fine-tuning, you likely only need the (image, correct_interpretation) pairs, not user identity. Consider whether synthetic handwriting generation could reduce the need for real user data.

—

Stretch Goals

1. **Confidence indicator for handwriting recognition.** When the LLM isn't sure about the handwriting, show the user what it "saw" (render the interpreted expression) and let them correct misrecognitions before getting feedback. Wrong feedback due to misrecognition is worse than no feedback.
2. **Offline problem bank with on-device storage.** Use SwiftData or CoreData to cache problem sets, solutions, and user progress locally. Students can review past problems without internet, and the app syncs to the server when connected. The LLM calls still need network, but browsing and review can work offline.

—

Discussion Notes

The team coordination is clear: each member owned a specific progress item (Sævar on observability, Sölvi on UI/UX, Jóhannes on data collection). This is good practice.

The new UI improvements look good. Aim for consistency in the pop-up modals – inconsistent styling in modals can feel unpolished to users.

The observation about thought tokens dominating usage is worth acting on. For an app like this, different operations have different reasoning requirements. Consider routing to different models: a fast model without thought tokens for hints, a reasoning model for checking multi-step solutions. This reduces both latency and cost for the most frequent operations.

Building in Swift gives you PencilKit for handwriting input, which is central to the product's value. The trade-off is a smaller user base (iOS only) and harder deployment (App Store review). For the course project this is fine, but if you wanted to reach more users, a web canvas with a stylus-friendly interface could replicate the core experience.

For the final submission, the "What the Data Tells You" section should be stronger. Even without real users, generate data from team testing. Fifty self-generated sessions would let you show the analytics pipeline producing real insights like "integration problems have a lower success rate than derivative problems" or "hint requests correlate with longer solution times."